Build Task — Config & Logging Foundation — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Where this Build Task is saved: its own folder, `practice/build_task/` — the reusable foundation every later document copies:

```
practice/build_task/
├── exceptions.py        MissingConfigError
├── config.py            load_config() -> Config
├── logging_setup.py     get_logger(name)
├── test_config.py       proves the 4 Test Cases from the README
├── .env.example         key names only, no real values (committed)
└── .env                 your real values (never committed)
```

Run it: `cd practice/build_task && python test_config.py` — from inside the folder, so `from config import load_config` finds the file next to it.

Builds on: `practice/config_logging_wiring/` (Real-world exercise) — **copy** that whole folder to `practice/build_task/`, then upgrade it: typed `Config`, a `.env.example`, a duplicate-handler guard in `get_logger()`, and `test_config.py` in place of `main.py`.

Used later by: every later document imports these two files — `practice/build_task/config.py` (`load_config()`) and `practice/build_task/logging_setup.py` (`get_logger(name)`). Doc02 is the first to say so out loud, and copies both into its own project folder. The names `load_config()`, `Config`, `MissingConfigError` and `get_logger(name)` are fixed for exactly that reason.

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

You're building two small, separate things that get used together: a **config loader** (`practice/build_task/config.py`) and a **logger factory** (`practice/build_task/logging_setup.py`). Keep them as two separate files, doing two separate jobs.

Think about the config loader first. Its whole job is: read some environment variables, and either hand back an object you can trust, or fail loudly with a clear reason. The key idea is **fail at startup, not later** — if `OPENAI_API_KEY` is missing, you want to know *immediately*, with a message that names the missing key, not five minutes later as a confusing `AuthenticationError` from OpenAI.

For the logger, the idea is: one function, `get_logger(name)` , that any file can call to get a working logger. Don't overthink this one — Python's built-in `logging` module already does almost everything you need.

Here's what to actually go look at:

- **`os.getenv(key)`** — returns the value, or `None` if the key isn't set. This is your basic tool for reading environment variables. `os.environ[key]` is the alternative, but it raises `KeyError` instead of returning `None` — decide which behavior you want to build on top of.
- **`python-dotenv`'s `load_dotenv()`** — call this once, near the top of your program, before you read any environment variables. It reads `.env` and loads its contents into `os.environ` for you.
- **A dataclass** (`from dataclasses import dataclass`) is a clean way to build your `Config` object — it gives you a typed object with almost no boilerplate.
- **`logging.getLogger(name)`** always returns the *same* logger object if called twice with the same name — Python caches them for you. You don't need to build your own caching.
- **`logging.StreamHandler()`** writes to the console; **`logging.FileHandler(path)`** writes to a file. Each handler can have its own `.setLevel(...)` , independent of the logger's own level.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

`practice/build_task/exceptions.py`:

```
class MissingConfigError is an Exception
```

`practice/build_task/config.py`:

```
define a Config data holder with fields: openai_api_key, log_level

function load_config():
    call load_dotenv() to read the .env file into the environment
    read OPENAI_API_KEY from the environment
    if it is missing or empty:
        raise MissingConfigError, message naming "OPENAI_API_KEY"
    read LOG_LEVEL from the environment, default to "INFO" if not set
    return a Config built from these values
```

`practice/build_task/logging_setup.py`:

```
function get_logger(name):
    get (or create) a logger for this name
    if this logger doesn't already have a console handler attached:
        create a StreamHandler, set its level from an env var (default INFO)
        attach it to the logger
    set the logger's overall level low enough that the handler is
    the one that actually filters
    return the logger
```

The "if this logger doesn't already have a handler" check matters — without it, calling `get_logger("x")` twice would attach two handlers, and you'd see every log line printed twice.

Turning that plan into real code, here's the one piece worth seeing on its own first — this is *not* the full solution, just the trickiest part (checking a required key is genuinely missing, not just falsy):

```
# practice/build_task/config.py
import os

def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value
```

Use this same pattern for every required key in `load_config()`, instead of writing the same `if` check out by hand each time.

For the logger's duplicate-handler problem, the check looks like this:

```
# practice/build_task/logging_setup.py
if not logger.handlers:
    # only attach a handler the first time this logger is configured
    ...
```

Try finishing the rest yourself before looking at the full Solution below.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

Story — `.env.example` / `.env`: every project needs somewhere to put a real API key, and that place can never be a file git tracks with the real value in it. `.env.example` is the checked-in template (key names, no values); `.env` is your own copy, gitignored. **If not:** the first time someone forgets to gitignore `.env`, a real key ends up in a public repo's history forever, even if you delete it in a later commit.

```
# practice/build_task/.env.example
# Why: shows every teammate which keys to set, without leaking real values into
# git.
# Copy this file to .env and fill in the real values. Never commit .env.
OPENAI_API_KEY=
LOG_LEVEL=INFO
```

```
# practice/build_task/.env
OPENAI_API_KEY=sk-test-123
```

Story — `exceptions.py`: `load_config()` needs a way to say "something required is missing" that's more specific than a bare `Exception`. `MissingConfigError` is that one purpose-built signal. **If not:** every caller that wants to catch a config problem would have to catch `Exception` broadly, which also silently swallows real bugs (a typo, a `NameError`) that have nothing to do with missing settings.

```
# practice/build_task/exceptions.py
# Why: gives load_config() its own error type, so calling code can catch a
# missing
# setting specifically, instead of catching every possible Exception blindly.
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

Story — `config.py`: this is the one place the whole project reads its settings from. Written this way — fail loudly, at startup, from one function — so a missing `OPENAI_API_KEY` is a clear one-line error the moment the program starts, not a confusing `AuthenticationError` from OpenAI five minutes into a run. **If not:** every file that needs a setting would read `os.environ` directly, so the same missing-key bug would need fixing in N different places instead of one, and a typo'd key name would fail silently with `None` instead of a clear error.

```
# practice/build_task/config.py
# how: os.getenv() is how we read env vars
import os
# why: @dataclass auto-builds __init__ for a typed holder
from dataclasses import dataclass
# how: reads .env and loads it into os.environ
from dotenv import load_dotenv
# why: a specific error type, not a bare Exception
from exceptions import MissingConfigError

@dataclass
class Config:
    # why: a typed object, not a plain dict — your editor autocompletes
    # .openai_api_key and .log_level, and catches a typo'd field name.
    openai_api_key: str
    log_level: str

def require_env(key: str) -> str:
    # why: one place that turns a missing/empty env var into a clear, named
    # error,
    # instead of repeating the same if-check for every required key by hand.
    # when: called once per required key, inside load_config(), before anything
    # else in the program runs.
    value = os.getenv(key)
    if value is None or value == "":
        # how: both "never set" (None) and "set but blank" (empty string)
        # count as missing — an empty API key is never usable either way.
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config() -> Config:
    # why: the one function every other file calls to get settings — so a
    # missing
    # key fails loudly here, at startup, instead of crashing confusingly later.
    # when: call this once, near the top of main()/test_config.py, before
    # anything that needs a setting runs.
    # how: reads .env into os.environ; must run before os.getenv() below
```

```
    load_dotenv()
    # how: required — raises if missing
    api_key = require_env("OPENAI_API_KEY")
    # how: optional — "INFO" if unset
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)
```

Story — `logging_setup.py`: every script needs to print progress and errors somewhere, and it should look the same everywhere in the project — same timestamp format, same way of separating "quiet console" from "detailed file." `get_logger(name)` is that one shared setup. **If not:** every file would configure its own `logging.StreamHandler()` by hand, some would forget the duplicate-handler guard and print every line twice, and log output would look different from file to file.

```
# practice/build_task/logging_setup.py
import logging
import os

def get_logger(name: str) -> logging.Logger:
    # why: gives every file in the project the same logging setup from one
    # place,
    # instead of each file configuring handlers/levels its own way.
    # when: call once per module, at the top — logging.getLogger(name) always
    # returns the same object for the same name, so calling it again elsewhere
    # in the same run is safe and cheap.
    logger = logging.getLogger(name)

    if not logger.handlers:
        # how: only attach a handler the first time this name is configured —
        # without this guard, calling get_logger(name) twice would print every
        # line twice.
        level_name = os.getenv("LOG_LEVEL", "INFO")
        # how: falls back to INFO if the env value is invalid
        level = getattr(logging, level_name.upper(), logging.INFO)

        # how: prints to the console
        handler = logging.StreamHandler()
        handler.setLevel(level)
        # why: every line carries a timestamp/source/level, not just the bare
        # message
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        formatter = logging.Formatter(log_format)
        handler.setFormatter(formatter)

        logger.addHandler(handler)
        # how: the logger's own level must be at least this low, or the handler
        # never even sees the message
```

```
        logger.setLevel(level)

    return logger
```

Story — `test_config.py`: `config.py` and `logging_setup.py` are about to get imported by every document in this curriculum — a silent regression here (say, the duplicate-handler guard breaking) would show up as a confusing bug three documents later, far from its real cause. This script runs all 4 Test Cases in one go, right here, right after writing the code. **If not:** you'd only find out `get_logger()` prints twice, or `load_config()` doesn't actually fail on a missing key, whenever some future document happens to trigger it — much harder to trace back to this file.

This version deliberately stays at the same level as the rest of this document — plain `os` functions, no `pathlib`, no `try/finally`. It reads top to bottom, in the order things actually happen.

```
# practice/build_task/test_config.py
# Runs the four Test Cases from the README
# against config.py and logging_setup.py.
# Run it from inside this folder: python test_config.py
# It rewrites .env as it goes, so it backs your real one up first, and
# puts it back at the very end.
import os

from config import load_config
from exceptions import MissingConfigError
from logging_setup import get_logger

ENV_PATH = ".env"
BACKUP_PATH = ".env.backup"

def set_env_file(contents: str | None) -> None:
    # Puts .env into a known state for one case, and clears os.environ first.
    # why clear os.environ: load_dotenv() never overwrites a variable that's
    # already set, so without this, case 2 would still see case 1's values.
    os.environ.pop("OPENAI_API_KEY", None)
    os.environ.pop("LOG_LEVEL", None)
    if contents is None:
        if os.path.exists(ENV_PATH):
            os.remove(ENV_PATH)
    else:
        with open(ENV_PATH, "w") as env_file:
            env_file.write(contents)

def main() -> None:
    # Step 1: hide your real .env somewhere safe, so the fake ones below
    # can't touch it. Nothing is deleted — just renamed out of the way.
    if os.path.exists(ENV_PATH):
```

```
        os.rename(ENV_PATH, BACKUP_PATH)

    # Case 1: no .env file at all. load_config() should fail clearly.
    set_env_file(None)
    try:
        load_config()
    except MissingConfigError as e:
        print(f"case 1 (.env missing)     -> MissingConfigError: {e}")

    # Case 2: .env exists, but OPENAI_API_KEY isn't in it. Should still fail.
    set_env_file("LOG_LEVEL=INFO\n")
    try:
        load_config()
    except MissingConfigError as e:
        print(f"case 2 (key missing)      -> MissingConfigError: {e}")

    # Case 3: .env has everything it needs. Should succeed this time.
    set_env_file("OPENAI_API_KEY=sk-test-123\n")
    config = load_config()
    log_level = config.log_level
    print(f"case 3 (valid .env)       -> Config loaded, log level {log_level}")

    # Case 4: calling get_logger() twice with the same name should not
    # attach a second handler — the line below should print once, not twice.
    get_logger("x")
    logger = get_logger("x")
    logger.info("case 4 (get_logger twice) -> printed exactly once")

    # Step 5: done testing — delete the fake .env, bring your real one back.
    if os.path.exists(ENV_PATH):
        os.remove(ENV_PATH)
    if os.path.exists(BACKUP_PATH):
        os.rename(BACKUP_PATH, ENV_PATH)

if __name__ == "__main__":
    main()
```

Expected output (the exact timestamp will differ on your machine — `%(asctime)s` always prints the current time):

```
case 1 (.env missing)     -> MissingConfigError:
Required environment variable is missing: OPENAI_API_KEY
case 2 (key missing)      -> MissingConfigError:
Required environment variable is missing: OPENAI_API_KEY
case 3 (valid .env)       -> Config loaded, log level INFO
2026-09-10 09:00:00,123 x INFO case 4 (get_logger twice) -> printed exactly once
```

(Cases 1 and 2 are shown wrapped onto two lines here just to fit the page — each is really one line of real output.) The last line is the only one that goes through the logger, so it's the only one carrying the timestamp/name/level prefix — and it appears once, not twice, which is the whole point of the `if not logger.handlers:` guard. One thing that surprises people: a `StreamHandler` writes to **stderr**, while `print()` writes to stdout. In a terminal they interleave in the order shown; pipe the output to a file (`python test_config.py > out.txt`) and the logger line can jump ahead of the others, because the two streams are buffered differently. Nothing is wrong when that happens.

Why this approach: a `@dataclass` gives you a typed, readable `Config` object with almost no extra code. This is a very common, standard pattern in real Python projects.